Module 2: Introduction to React

---

1. What is React?

- **Definition**: React is a **JavaScript library** used for building user interfaces (UI), specifically for single-page applications. It allows developers to create reusable UI components and manage the state of an application efficiently.
- **Key Features of React**:
  - **Component-Based**: The UI is divided into small, reusable components.
  - **Declarative**: You describe what you want the UI to look like, and React takes care of updating it when the data changes.
  - **Virtual DOM**: React uses a virtual DOM to optimize rendering and improve performance.

Example Problem:

Problem: What would happen if you wanted to display a list of items in HTML, and then needed to update this list dynamically when new items are added?

Solution (Traditional JavaScript):

```
<ul id="itemList">
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
<script>
  const newItem = document.createElement("li");
  newItem.textContent = "Item 3";
  document.getElementById("itemList").appendChild(newItem);
</script>
```

Solution (React):

```
import React, { useState } from 'react';

function ItemList() {
  const [items, setItems] = useState(["Item 1", "Item 2"]);

  const addItem = () => {
    setItems([...items, `Item ${items.length + 1}`]);
  };

  return (
    <div>
      <ul>
        {items.map(item => <li key={item}>{item}</li>)}
      </ul>
      <button onClick={addItem}>Add Item</button>
    </div>
```

```
  );
}

export default ItemList;
```

Explanation: React automatically updates the UI when the state (`items`) changes. You don't have to worry about manually manipulating the DOM.

---

2. Why use React?

- **Efficiency**: React is designed to make UI updates fast, using its Virtual DOM to minimize the actual changes to the real DOM.
- **Component Reusability**: Components are reusable pieces of code, making it easier to build complex UIs and maintain them.
- **Declarative Syntax**: React allows developers to describe how the UI should look at any given point in time, and React will handle updating it when the state or data changes.
- **Large Ecosystem**: React has a massive community and ecosystem of libraries, tools, and tutorials, making development easier.

Example Problem:

Problem: You need to create a counter that increases when a button is clicked.

Solution (Using React):

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </div>
  );
}

export default Counter;
```

Explanation: Here, React helps manage the state (`count`) and automatically updates the UI whenever the state changes. The `useState` hook is used to handle the state in a functional component.

---

3. React vs Traditional JavaScript

- **Traditional JavaScript**:
  - You manually manipulate the DOM using `document.getElementById()` or similar methods.
  - Updating the UI can become complex as the application grows.
- **React**:
  - React abstracts away the direct manipulation of the DOM. You just need to define how the UI should look, and React will handle updates for you.
  - React components can be reused across different parts of the app, making the codebase more maintainable.

Example Comparison:

Traditional JavaScript (DOM Manipulation):

```
<button id="changeText">Change Text</button>
<p id="text">Hello, World!</p>

<script>
  document.getElementById('changeText').onclick = function() {
    document.getElementById('text').innerHTML = "New Text!";
  }
</script>
```

React Version:

```
import React, { useState } from 'react';

function App() {
  const [text, setText] = useState("Hello, World!");

  return (
    <div>
      <p>{text}</p>
      <button onClick={() => setText("New Text!")}>Change Text</button>
    </div>
  );
}

export default App;
```

Explanation: In React, you don't manually change the content of the `<p>` tag. React handles UI updates automatically based on state changes.

---

4. Setting up the Environment

a. Node.js & npm

- **Node.js**: A JavaScript runtime environment that allows you to run JavaScript on the server side. It's essential for running React development tools like Create React App.
- **npm**: A package manager for JavaScript, used to install libraries and tools (including React).

Task: Install Node.js and npm

- Go to Node.js official website and download the latest stable version.
- After installing, check if Node.js and npm are installed by running these commands in the terminal:
- `node -v`
- `npm -v`

b. Create React App

- **Create React App** is an officially supported way to create React applications with zero configuration. It sets up everything you need (like webpack, Babel, etc.) to start coding React right away.

Task: Set up a new React project

- Open a terminal or command prompt.
- Run the following command to create a new React app:
- `npx create-react-app my-app`

  This will create a folder called `my-app` with all the necessary files and dependencies.

- Navigate to the new project folder:
- `cd my-app`
- To run the app, use:
- `npm start`

Task: Modify the default React app

- Open the `src/App.js` file and change the text inside the `<h1>` tag to "Welcome to React!"

---

5. Understanding Project Structure

Once you've created a new React app using `create-react-app`, you'll see the following folder structure:

```
my-app/
├── node_modules/
├── public/
```

```
│   └── index.html
└── src/
    ├── App.css
    ├── App.js
    ├── index.css
    └── index.js
```

- **`public/`**: Contains the `index.html` file. React apps typically load a single HTML file and use JavaScript to update the content dynamically.
- **`src/`**: Contains your React components and other files related to your app's logic.
  - `App.js`: The main React component.
  - `index.js`: The entry point for the React application. This is where the app is rendered into the HTML.

Task: Explore the `src/App.js` file

- Open the `src/App.js` file and try modifying the default content (e.g., change the text inside the `<h1>` tag to something else).
- Refresh the browser to see the changes.

Module 3: JSX & Components

---

1. What is JSX?

JSX (JavaScript XML) is a syntax extension for JavaScript that allows you to write HTML-like code within JavaScript files. It makes it easier to create and manage React elements and components. JSX is then transformed into regular JavaScript by tools like Babel.

Key Points about JSX:

- **HTML-like Syntax**: JSX looks very similar to HTML but has the power of JavaScript behind it.
- **JavaScript Expressions**: You can embed JavaScript expressions in JSX using curly braces `{}`.
- **Must be transpiled**: JSX cannot run directly in the browser. It must be converted into JavaScript via a tool like Babel.

Example Code (JSX):

```
const element = <h1>Hello, World!</h1>;
```

This JSX will be transformed into JavaScript:

```
const element = React.createElement('h1', null, 'Hello, World!');
```

In React, JSX is not just for writing static HTML, but can include dynamic content, loops, and even functions.

---

2. JSX Rules

To make sure JSX works correctly, there are a few rules and best practices to follow.

JSX Rules:

1. **Must return a single element**: JSX code must return a single element. If you want to return multiple elements, you should wrap them in a parent element (like a `<div>` or `<React.Fragment>`).

```
// Invalid JSX (Multiple top-level elements)
return (
  <h1>Hello</h1>
  <p>World</p>
);
```

Solution: Wrap in a parent element:

```
return (
  <div>
    <h1>Hello</h1>
    <p>World</p>
  </div>
);
```

7. **HTML attributes use camelCase**: In JSX, attributes like `class` and `for` (in HTML) are replaced with `className` and `htmlFor` to avoid conflicts with JavaScript reserved words.

```
<div className="container">
  <label htmlFor="name">Name:</label>
    <input id="name" />
  </div>
```

12. **JavaScript expressions in JSX**: Any JavaScript expression can be included in JSX using curly braces `{}`.

```
const name = 'John';
return <h1>Hello, {name}!</h1>;
```

15. **Self-closing tags**: If an HTML element doesn't have children, you must close the tag using a self-closing syntax.

```
<img src="image.jpg" alt="image" />
```

17. **JSX is case-sensitive**: JSX tags must start with a capital letter for custom components and lowercase for HTML elements.

```
// Correct
<MyComponent />
<div />

// Incorrect
<myComponent />
```

---

3. Functional Components

A **functional component** is a JavaScript function that returns JSX and represents a part of the UI. It accepts **props** (inputs) and returns a React element.

Example of a Functional Component:

```
import React from 'react';

function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

export default Greeting;
```

- **`props`**: The function accepts `props` as a parameter. This allows the component to receive dynamic data from its parent component.

Using the Functional Component:

```
import React from 'react';
import Greeting from './Greeting';

function App() {
  return (
    <div>
      <Greeting name="John" />
      <Greeting name="Alice" />
    </div>
  );
}

export default App;
```

Here, the `Greeting` component is being reused with different `name` props.

---

4. Component Structure

A React component typically follows this structure:

1. **Import React**: Always import React (and `useState`, `useEffect`, etc. if needed).
2. **Component Definition**: A functional or class-based function that returns JSX.
3. **Exporting the Component**: To use the component elsewhere, it must be exported.
4. **Props & State**: Components can accept `props` and manage internal `state`.

Example of a Complete Component:

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0); // State to keep track of count

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(count - 1)}>Decrement</button>
    </div>
  );
}

export default Counter;
```

This is a functional component with state, handling click events to update the count.

---

5. Exporting & Importing Components

- **Exporting**: When you want to use a component in another file, you export it. In React, there are two main ways to export:
  - **Named Export**: You can export multiple components or functions from the same file.
  - **Default Export**: You can only have one default export per file.

Named Export:

```
// Button.js
export function Button() {
  return <button>Click Me</button>;
}

// App.js
import { Button } from './Button';
```

Default Export:

```
// Button.js
export default function Button() {
  return <button>Click Me</button>;
}
```

```
// App.js
import Button from './Button'; // No curly braces needed
```

Example Program:

```
// App.js
import React from 'react';
import Greeting from './Greeting'; // Import the Greeting component

function App() {
  return (
    <div>
      <Greeting name="John" />
      <Greeting name="Alice" />
    </div>
  );
}

export default App;

// Greeting.js
import React from 'react';

function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

export default Greeting;
```

Practice Task Questions

1. **Task 1: Create a "Todo List" component**
   - Create a functional component called `TodoList`.
   - Inside it, create a list of tasks (`To Do 1`, `To Do 2`, etc.) using JSX.
   - Display the list using `map()` and use a `key` for each element.
   - Bonus: Add an `addTodo()` function to allow adding new tasks dynamically.
2. **Task 2: Create a "User Profile" component**
   - Create a functional component `UserProfile` that takes `name`, `age`, and `location` as props.
   - Display the information in a styled `<div>` (you can use CSS for styling).
   - Use this component inside your `App` component, passing in some sample data for the props.
3. **Task 3: Build a simple counter with increment and decrement buttons**
   - Create a component `Counter` that displays a number and has two buttons: one for incrementing the count and one for decrementing it.
   - Use React's `useState` hook to manage the count.
   - Display the current count in a `<p>` tag.